

Streaming Protocol Explanation

Overview:

Audio streaming requires delivering audio data to clients in a way that enables playback to start quickly while the file is still being downloaded. There are several approaches:

Approach 1: HTTP Range Requests (Byte-Range Requests)

Description:

- Standard HTTP feature that allows clients to request specific byte ranges of a file
- Server responds with HTTP 206 Partial Content status
- Client can request multiple ranges in parallel for faster buffering

Example Flow:

- Client requests: GET /api/stream/song123?start=0&end=1048575
Headers: Range: bytes=0-1048575
- Server responds: HTTP 206 Partial Content
Headers: Content-Range: bytes 0-1048575/5242880
Body: First 1MB chunk of audio file
- Client starts playing while requesting next chunk: Range: bytes=1048576-2097151
- Process continues until entire file is streamed

Benefits:

- Simple to implement (standard HTTP)
- Works with any web server/CDN
- No special client libraries needed
- Supports seeking (jump to any position)

- Efficient for on-demand streaming

Limitations:

- Client must manage chunk requests and buffering
 - No automatic quality adaptation
 - Requires file to be available as a single file
-

Approach 2: HLS (HTTP Live Streaming)

Description:

- Apple's protocol that breaks audio into small segments (typically 10 seconds)
- Creates a playlist file (.m3u8) listing all segments
- Client downloads segments sequentially and plays them

Example Flow:

- Client requests: GET /api/stream/song123.m3u8
- Server returns playlist file:
#EXTM3U
#EXTINF:10.0,segment1.ts
#EXTINF:10.0,segment2.ts
...
- Client downloads segments sequentially: segment1.ts, segment2.ts, etc.
- Client plays segments as they arrive

Benefits:

- Automatic quality adaptation (multiple quality playlists)
- Built-in error recovery
- Works well for live and on-demand content
- Standardized protocol

Limitations:

- More complex to implement
 - Requires segmenting files in advance
 - Slight overhead from playlist files
-

Approach 3: DASH (Dynamic Adaptive Streaming Over HTTP)

Description:

- Similar to HLS but more flexible and standardized
- Uses XML manifest files instead of m3u8 playlists
- Supports multiple audio/video tracks and quality levels

Benefits:

- Very flexible and feature-rich
- Industry standard
- Excellent for adaptive bitrate

Limitations:

- Most complex to implement
 - Requires significant infrastructure
-

Selected Approach: HTTP Range Requests (Approach 1)

For interview scope, we use **HTTP Range Requests** because:

- **Simplicity:** Easy to understand and implement
- **Interview-friendly:** Demonstrates understanding of HTTP protocol
- **Sufficient:** Meets requirements for chunk-based streaming
- **Flexible:** Supports seeking and progressive playback

- **Standard:** Works with any HTTP client/server

Implementation Details:

- **Chunk Size:** 1MB (1048576 bytes) – configurable
- **Cache Key Format:** "chunk_{songId}_{start}_{end}"
- **HTTP Status:** 206 Partial Content for range requests
- **Headers:** Content-Range, Accept-Ranges: bytes
- **Client Behavior:** Request chunks sequentially or in parallel for buffering
- **Prefetching:** Client can request next chunk while current chunk is playing